

Core & Expected DSA Programming Questions

ARRAYS

1. **Find missing number** from an array of size n containing numbers from $1 \dots n+1$.
 2. **Find duplicates** in an array $O(n)$ time & $O(1)$ space (no extra DS).
 3. **Rotate an array by k positions** (in-place).
 4. **Merge two sorted arrays** into one sorted array (in place if possible).
 5. **Majority element (Boyer-Moore)** in $O(n)$ time.
 6. **Longest subarray with sum = k** (hashing + prefix sums).
-

STRINGS

1. **Reverse a string in place.**
 2. **Longest substring without repeating characters.**
 3. **Check if string is rotation of another** (use KMP/concat trick).
 4. **Count and say / anagram detection.**
 5. **Palindrome checks, longest palindrome.**
-

TWO POINTER / SLIDING WINDOW

1. **Container with most water.**
2. **Maximum sum subarray with constraints.**

3. **Backspace string compare.**
-

STACK / QUEUE

1. **Balanced parentheses** and variants.
 2. **Next greater element** and previous greater.
 3. **Implement queue using stacks.**
-

LINKED LIST

1. **Reverse a linked list** (iteratively/recursively).
 2. **Detect cycle (Floyd's algorithm).**
 3. **Merge two sorted lists.**
-

SEARCHING & SORTING

1. **Binary search & variants** (first/last occurrence).
 2. **Quick sort / Merge sort understanding** and implementation detail questions.
 3. **Kth largest element** (heap or Quickselect).
-

BIT MANIPULATION

1. **Find single non-repeated element** using XOR.
2. **Count set bits** efficiently.

3. **Check power of two.**
-

RECURSION & BACKTRACKING

1. **Generate permutations & combinations.**
 2. **N-Queens** problem.
 3. **Subset sum / backtracking basics.**
-

TREES & GRAPHS (Advanced, but sometimes asked)

1. **Tree traversals** (inorder, preorder, level order).
2. **BFS/DFS traversal on graphs.**
3. **Shortest path basics (Dijkstra).**

Sample Set You *Must Practice*

Below are some **specific problems based on the above**:

1. **Find missing number** in an array of size n with numbers 1 to $n+1$ ($O(n)$, $O(1)$)
2. **Find all duplicates** using in-place marking
3. **Rotate array by k steps**
4. **Longest substring without repeating characters**
5. **Container with most water**
6. **Balanced parentheses**

7. **Detect cycle in linked list**
8. **Binary search variants (first/last occurrence)**
9. **Single non-repeated element (XOR trick)**
10. **Backtracking: Permutations**
11. **Tree traversal: level order**
12. **Graph BFS/DFS basics**

ARRAYS

Q1. Find Missing Number

(VERY IMPORTANT – asked everywhere)

Question:

Given an array of size n containing numbers from 1 to $n+1$ with exactly one number missing, find the missing number.

Test case:

Input: $[2, 3, 1, 5]$

Output: 4

Solution:

XOR all numbers from 1 to $n+1$ and XOR all array elements.
Final XOR gives missing number.

Time: $O(n)$

Space: $O(1)$

Q2. Find Duplicate Elements Without Extra Space

(VERY IMPORTANT)

Question:

Given an array of size n with elements in range 0 to $n-1$, find duplicates in $O(n)$ time and $O(1)$ extra space.

Test case:

Input: $[1, 2, 3, 1, 3, 6, 6]$

Output: $1\ 3\ 6$

Solution:

Use index marking ($arr[abs(arr[i])] \text{ } *= \text{ } -1$).

Time: $O(n)$

Space: $O(1)$

Q3. Rotate Array by K Positions

(IMPORTANT)

Question:

Rotate an array to the right by k positions in-place.

Test case:

Input: $[1, 2, 3, 4, 5]$, $k=2$

Output: $[4, 5, 1, 2, 3]$

Solution:

Reverse whole array $\rightarrow$ reverse first $k \rightarrow$ reverse remaining.

Time: $O(n)$

Space: $O(1)$



STRINGS

Q4. Check if String is Palindrome

(EASY but COMMON)

Question:

Check whether a string is a palindrome.

Test case:

Input: "madam"

Output: Yes

Solution:

Two pointers from start and end.

Time: $O(n)$
Space: $O(1)$

Q5. Longest Substring Without Repeating Characters

(IMPORTANT – problem solving)

Question:

Find length of longest substring without repeating characters.

Test case:

Input: "abcabcbb"

Output: 3

Solution:

Sliding window + hash set.

Time: $O(n)$

Space: $O(n)$

LINKED LIST

Q6. Detect Cycle in Linked List

(VERY IMPORTANT – classic PSU)

Question:

Detect if a linked list contains a cycle.

Test case:

1 → 2 → 3 → 4 → 2

Solution:

Floyd's slow and fast pointer.

Time: $O(n)$

Space: $O(1)$

Q7. Reverse a Linked List

(VERY IMPORTANT)

Question:

Reverse a singly linked list.

Test case:

1 → 2 → 3 → NULL

Output: 3 → 2 → 1 → NULL

Solution:

Iteratively reverse pointers.

Time: $O(n)$

Space: $O(1)$

STACK / QUEUE

Q8. Balanced Parentheses

(IMPORTANT)

Question:

Check if parentheses are balanced.

Test case:

Input: "({[]})"

Output: Valid

Solution:

Stack push opening, pop on matching close.

Time: $O(n)$

Space: $O(n)$

Q9. Next Greater Element

(IMPORTANT)

Question:

For each element, find the next greater element to its right.

Test case:

Input: [4, 5, 2, 25]

Output: [5, 25, 25, -1]

Solution:

Monotonic decreasing stack.

Time: $O(n)$

Space: $O(n)$



SEARCHING & SORTING

Q10. Binary Search First & Last Occurrence

(IMPORTANT)

Question:

Find first and last occurrence of a target in sorted array.

Test case:

Input: [1, 2, 2, 2, 3], target 2

Output: 1, 3

Solution:

Modified binary search.

Time: $O(\log n)$

Space: $O(1)$



BIT MANIPULATION

Q11. Single Non-Repeating Element

(VERY IMPORTANT)

Question:

Every element appears twice except one. Find it.

Test case:

Input: [2, 3, 5, 4, 5, 3, 4]

Output: 2

Solution:

XOR all elements.

Time: $O(n)$

Space: $O(1)$



RECURSION / BACKTRACKING

Q12. Generate All Permutations

(IMPORTANT)

Question:

Generate all permutations of a string.

Test case:

Input: "ABC"

Output: ABC ACB BAC BCA CAB CBA

Solution:

Swap-based recursion.

Time: $O(n!)$

Space: $O(n)$ (recursion stack)

TREES

Q13. Level Order Traversal

(IMPORTANT)

Question:

Print binary tree level by level.

Test case:

```
  1
 /  \
2    3
```

Output: 1 2 3

Solution:

Queue-based BFS.

Time: $O(n)$

Space: $O(n)$